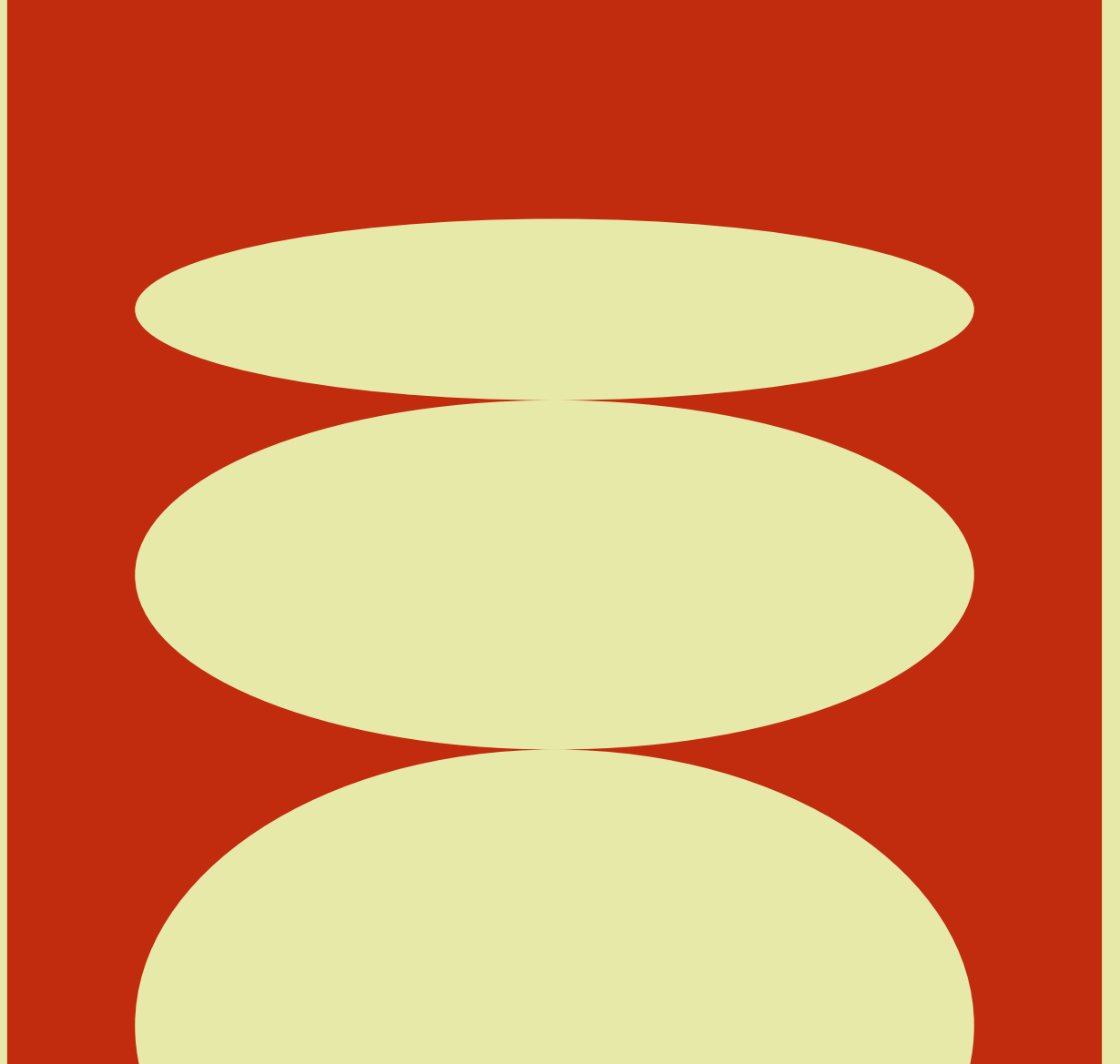


Presented by 21기 수습단원 이주원

Cross Road & Traffic light

Ros2 + Qt



Agenda

1. 프로젝트 개요
2. 노드 구성
3. Custom Msg / Srv 설계
4. 핵심 알고리즘
5. 예외 처리와 설정분리
6. 시연



1. 프로젝트 개요

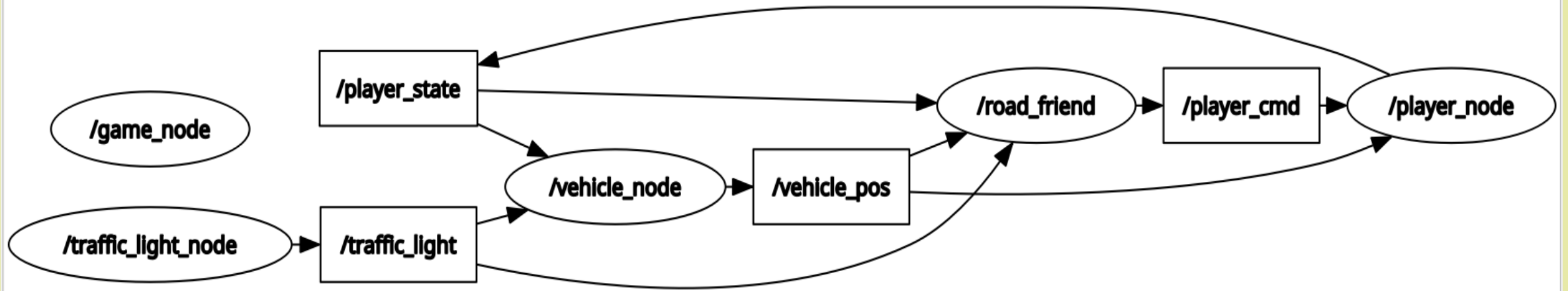
차를 피해 반대편까지 건너는 게임. ROS2
교육 3일차에 만든 신호등 시뮬레이터를
게임으로 확장

Qt는 화면만 담당, 신호 전환. 차량 이동. 충돌
판정 모두 노드에서 처리



2. 노드 구성

“노드 5개가
각자 한 가지 일만 한다”



타원 = 노드, 사각형 = 토픽

game_node는 서비스만 사용해
토픽 그래프에는 연결이 없다.

Node role

Node	Work	Publish
traffic_light	신호 순환 RED→GREEN→YELLOW	/traffic_light
vehicle	차량 생성 . 이동	/vehicle_pos
player	위치 관리 . 충돌 판정	/player_state
game	리셋 요청 처리 (서비스)	---
gui	화면 표시.버튼 입력	/player_cmd

3. Custom Msg / Srv 설계

**"Custom Msg = 4,
Srv = 1"**

Custom Msg / Srv 설계

Type	Content	Pub → Sub
CarArray	차선 + 위치 배열	vehicle → player, gui
Move	이동 변화량	gui → player
PlayerState	칸 · 생존 · 점수	player → gui, vehicle
Reset.srv	리셋 요청 / 시작위치	player ↔ game

- Move는 목적지가 아닌 변화량

4. 핵심 알고리즘

"같은 차선의 앞차를
한 번의 순회로 찾는다"

4. 핵심 알고리즘 - 앞차 간격

차선별 앞차 탐색

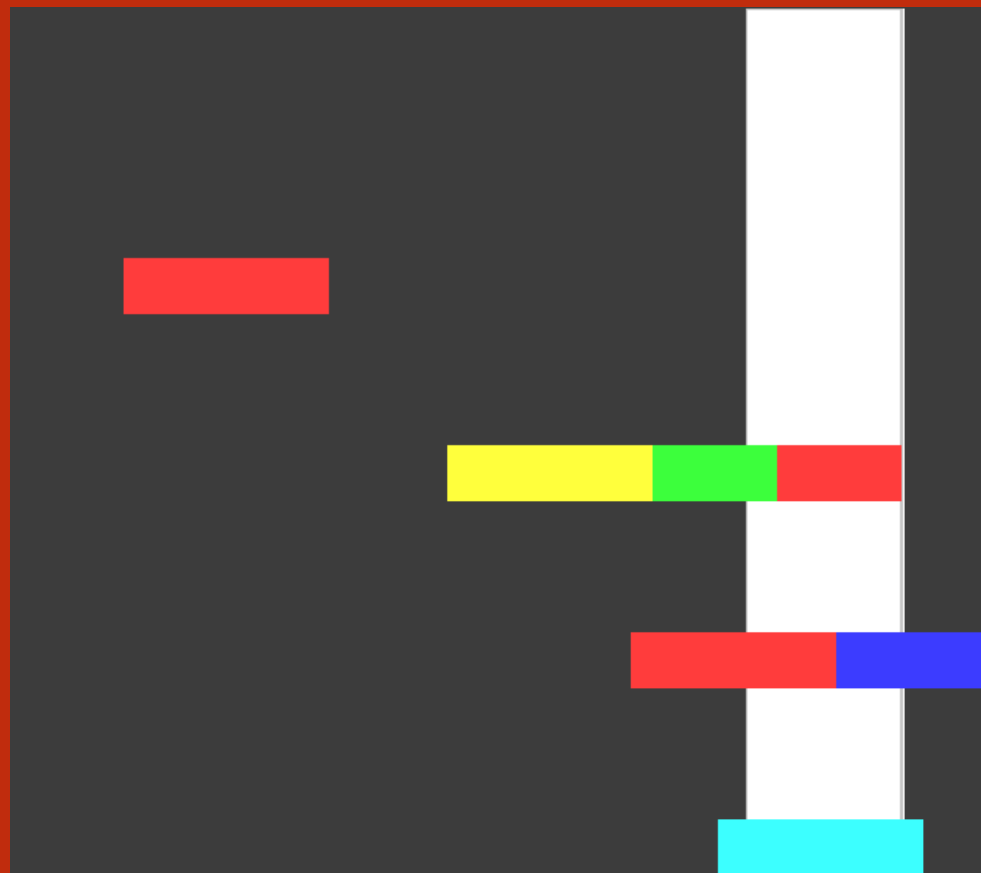
차선이 하나일 때는 배열의 바로 앞 인덱스가 곧 앞차. 차선이 섞이면서 그 방법이 통하지 않게 됨.

차선 개수만큼 칸을 둔 배열에 직전 차 위치를 기록하면, 한 번만 훑어도 각자의 앞차를 찾을 수 있음.

```
vehicles = [ {1차선,340}, {3차선,220}, {1차선,180} ]
```

↓ ↓ ↓

front = [-, 340, -, 220, -] → 3번째 차는 front[1]=340을 본다



4. 핵심 알고리즘 - 충돌 판정

충돌 판정

같은 차선인지 먼저 거르고,
가로 구간이 겹치는지만 확인한다.

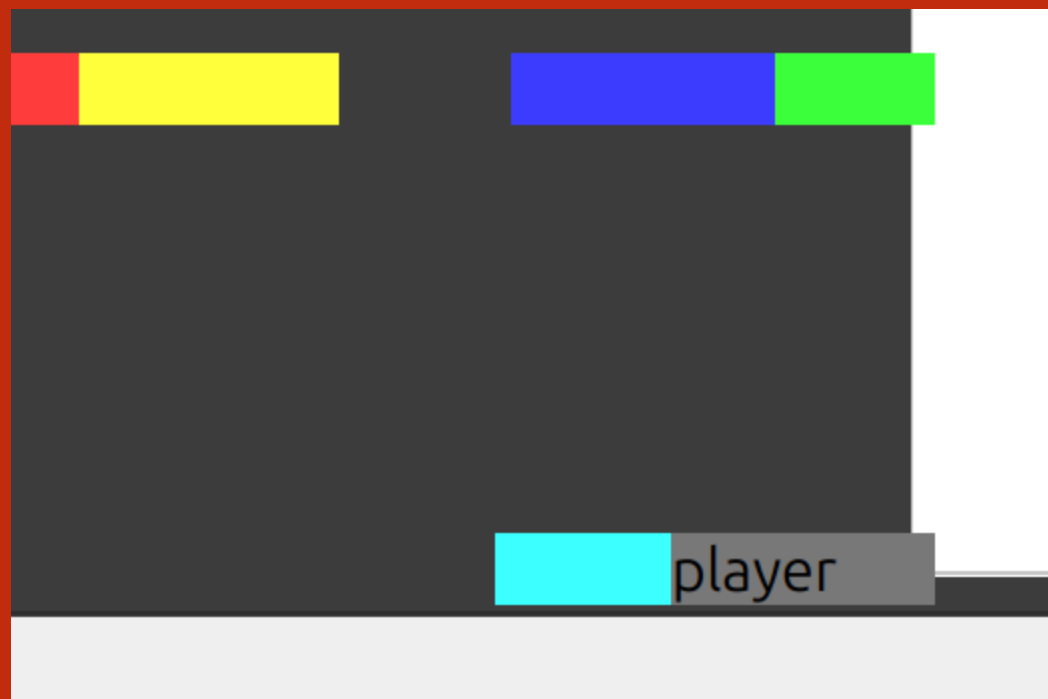
플레이어 [=====]

차 [=====]

↑ 겹침

플레이어 끝 > 차 시작

플레이어 시작 < 차 끝



5. 예외처리와 설정분리

"좀 더 완벽한 길로 가는법"

5. 예외처리와 설정 분리

예외 처리와 설정 분리

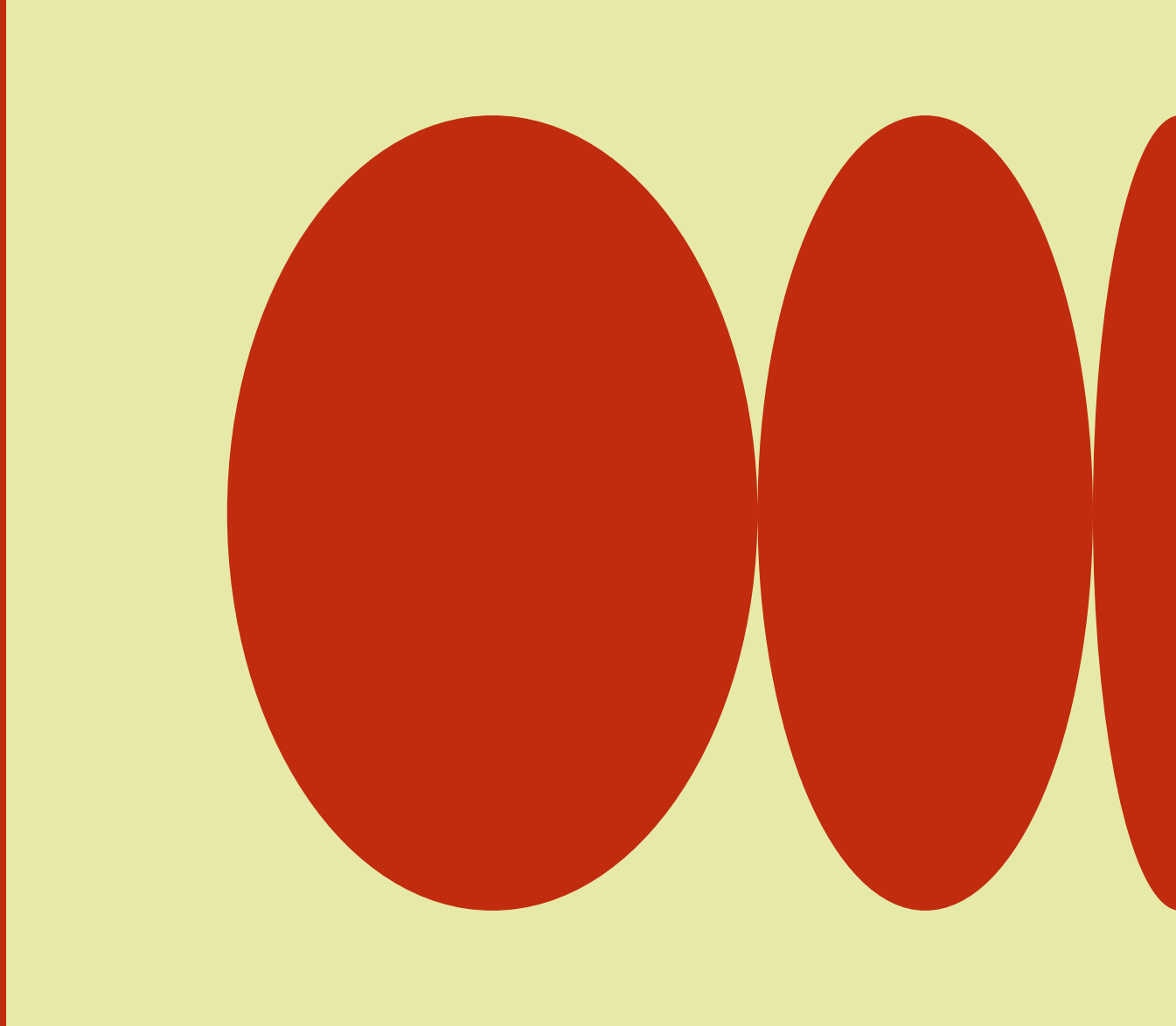
이동 요청은 후보값을 만들어 범위를 검사한 뒤

반영한다. 죽은 상태에서는 입력을 무시하고, 서비스는 호출 전에 가용성을 확인한다.

차선 수, 속도, 신호 시간은 yaml에서 관리해 코드 수정 없이 난이도를 조절할 수 있다.



6. 시연



End